CodeInspector - Project Requirements

19 October 2026 | Document revision 0.1.1

Software Construction and Development, IMSciences

Instructor: Engr. Muhammad Umer Haroon

Project and Member

CodeInspector will review Python student projects for clean code and completeness.

Abdul Samad | BSSE, Group A | Semester 5 | Roll number 2

GitHub: abdulsamad-codes

Repository: https://github.com/abdulsamad-codes/CodeInspector

This is an individual project with instructor permission. I will handle requirements, coding, testing, documentation, and Git updates.

Problem and Purpose

A program may work but still have confusing names, large functions, inconsistent formatting, or missing documentation. CodeInspector will help students notice these issues and give instructors a starting point for review.

Sir Umer suggested researching PEP 8 when reviewing the idea. The project will use selected PEP 8 style guidance alongside the clean code rules from the course slides.

What the Tool Should Do

1. Accept a local project folder and find Python files, skipping Git data, virtual environments, and generated files. Avoid following links outside that folder.

2. Read and parse files without running or importing them. Explain invalid paths, syntax errors, and unreadable files; continue with other files where possible.

3. Check selected PEP 8 naming rules: snake_case functions/variables, CapWords classes, lowercase modules/packages, and uppercase constants where identifiable. Respect special names and conventional self/cls arguments.

4. Check selected formatting rules: four-space block indentation, line lengths, blank lines, whitespace, and simple import layout. Start with 79-character code lines and 72-character comment/docstring lines, allowing configuration and valid continuation-line exceptions.

5. Apply course rules about short functions, shallow nesting, argument counts, multiple returns, loop control, and direct recursion. Review tail-call patterns where feasible. These are separate from PEP 8.

6. Identify missing README/test files and empty functions. Recognize common README formats and test_*.py or *_test.py files.

7. Produce a readable report with the rule, file, line where relevant, and a suggestion. Separate PEP 8, course, and completeness findings. Clearly report files that could not be checked.

8. Allow supported checks to be disabled and thresholds changed through a settings file. Reject invalid settings clearly.

9. Later, add unused-import and duplicate-function checks, report export, and scores with an explained calculation. Disabled checks must not reduce scores.

10. After the core works, research AI-use detection. Evaluate false positives before including it; any result must be labelled experimental rather than proof of authorship.

The first interface will be a local command-line tool. AST will handle structure; source text and tokens will support formatting checks.

For course checks, initial configurable limits are 20 physical lines per function, nesting depth 2 including the function body, and 2 explicit parameters excluding self/cls. The 20-line limit is my project choice, not a number specified in the slides. Nested functions will be checked separately.

Quality Requirements

- Keep analysis local and leave the input files unchanged.

- Use separate, focused functions for reading files, checking rules, and reporting.

- Give understandable errors and consistent results for the same input/settings.

- Target Python 3.11 or newer initially and avoid machine-specific paths.

- Test valid examples, violations, boundary cases, exceptions, and the guarantee that target code is never executed.

- Document supported checks and actual usage as development progresses.

Priorities and Responsibilities

First complete planning, then file reading/parsing, naming and formatting checks, course rules, completeness checks, settings, and reports. Unused imports, duplicate checks, exports, and scoring follow. AI-use detection comes last as research.

I own all tasks. The current task list is in docs/backlog.md. A task is done when its checks pass, its documentation is updated, and a meaningful commit records the work.

Assumptions and Limits

The tool is intended for small, readable Python student projects. Syntax support depends on the Python interpreter used. A web interface, database, and cross-team plagiarism detection are outside the initial scope.

Static checks cannot prove correct logic, meaningful names, or good design. Test files do not prove coverage. Dynamic Python behavior may cause missed findings or false positives. A score only summarizes the checks performed; incomplete scans must be marked clearly.

PEP 8 does not ban every short name or set limits on function size and argument counts. Those extra checks come from the course/project. Python does not optimize tail recursion. AI-associated style signals cannot reliably establish authorship.

Repository and Submission

Use the naming and version conventions in README.md. Keep actual work in meaningful commits and tag significant milestones instead of saving duplicate versions. v0.1.0 marks the planning baseline; the application is not implemented yet.

Update this file and docs/requirements.pdf together as requirements change. Review the work before submitting the public repository URL and the exact committed PDF. Keep the backlog and development history current throughout the semester.

References

- Assignment_1_SCD_IMSCIENCES.pdf, pages 2-3, and CodeInspector_Project_Proposal.pdf.

- Official PEP 8: https://peps.python.org/pep-0008/

- Course slides: Lecture 1 (naming), Lecture 2 (reuse and small functions), Lecture 2b (arguments, returns, and recursion), Lecture 3 (releases and versions), and Lecture 4 (text patterns).